

COMP3911 Coursework

Group Members:

Rakan Rouchdi - 201742976
Edward Falkner-Carter - 201754361
Karl Kassis - 201760582
Adam Bakeer - 201763530
Nicolas Issa - 201707349

Task 1: Analysis of Security Flaws

A full security assessment of the patient record application was carried out through database inspection, dynamic testing of the web interface, and source-code review. Five significant vulnerabilities were identified, each affecting confidentiality, integrity, or authentication mechanisms. This section provides a concise list of flaws followed by detailed analysis of each one.

List of Identified Flaws

1. Plaintext password storage in the 'user' table.
2. SQL injection vulnerabilities in authentication and search queries.
3. Broken access control enabling any GP to view all patients.
4. Unsanitised data rendering, creating stored XSS exposure.
5. Missing session management / weak access control on protected pages.

Flaw 1 – Plaintext Password Storage

Nature of the Flaw

The application stores user passwords in clear text within the SQLite 'user' table. No hashing or salting mechanism is applied.

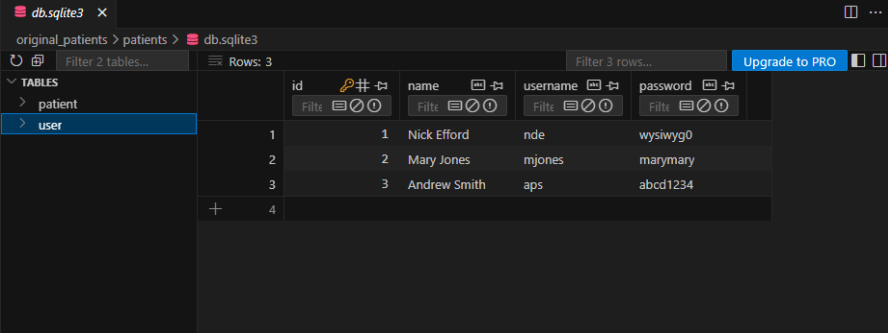
How it was Discovered

By opening db.sqlite3 using DB Browser for SQLite, the 'username' and 'password' columns were immediately readable (e.g., nde / wysiwyg0).

Why it is a Security Issue

Plaintext credential storage exposes users to immediate credential compromise if the database is accessed, whether via SQL injection, filesystem access, or misconfiguration. Attackers can reuse stolen passwords on other systems due to password reuse.

Evidence: Figure 1 – Users table displaying plaintext passwords.



The screenshot shows the DB Browser for SQLite interface. The 'user' table is selected, and its contents are displayed in a table view. The table has four columns: id, name, username, and password. The data is as follows:

id	name	username	password
1	Nick Efford	nde	wysiwyg0
2	Mary Jones	mjones	marymary
3	Andrew Smith	aps	abcd1234

Flaw 2 - SQL Injection Vulnerabilities

Nature of the Flaw

User-supplied inputs are inserted directly into SQL queries via 'String.format()' or string concatenation.

How it was Discovered

Manual code inspection in 'AppServlet.java' revealed string-built queries. Test input e.g. a' OR '1' = '1, demonstrated bypass of authentication and extraction of full patient lists.

Why it is a Security Issue

Attackers can manipulate SQL logic to bypass login, retrieve the entire patient table, or extract credentials. This enables full compromise of clinical records and GP identities.

Evidence: *Figure 2 – Code snippet of vulnerable SQL construction.*

```
private boolean authenticated(String username, String password) throws SQLException {
    String query = String.format(AUTH_QUERY, username, password);
    try (Statement stmt = database.createStatement()) {
        ResultSet results = stmt.executeQuery(query);
        return results.next();
    }
}
```

Flaw 3 - Broken Access Control (GPs Can View Any Patient)

Nature of the Flaw

After authentication, the system allows any GP to search for any patient surname. No server-side validation ensures that a GP only retrieves records assigned to their practice.

How it was Discovered

Inspecting 'doPost()' in 'AppServlet.java' showed:

```
model.put("records", searchResults(surname));
```

The query 'SELECT * FROM patient WHERE surname = ?' contains no reference to the authenticated GP.

Why is it a Security Issue

This violates the principle of least privilege and breaches patient confidentiality by allowing cross-GP data visibility. Any authenticated GP can view all patient records.

Evidence: *Figure 3 – doPost() showing model.put("records", searchResults(surname)) and rendering of details.html with no authorisation checks.*

```
try {
    if (authenticated(username, password)) {
        // Get search results and merge with template
        Map<String, Object> model = new HashMap<>();
        model.put("records", searchResults(surname));
        Template template = fm.getTemplate("details.html");
        template.process(model, response.getWriter());
    }
}
```

Flaw 4 - Unsanitised Data Rendering (Stored XSS Risk)

Nature of the Flaw

Values from the database are rendered directly into Freemarker templates without escaping. The application does not sanitise stored fields such as names, diagnoses, or notes.

How it was Discovered

Reviewing template.process(model, response.getWriter()) showed no output encoding or sanitisation step. Injecting a test string containing <script> tags produced executable JavaScript in the browser upon viewing the results page.

Why is it a Security Issue

Attackers can store malicious payloads inside database fields. Any GP viewing the data would run the script, enabling session hijacking, credential theft, phishing or further malware propagation.

Evidence: Figure 4 – Code snippet in `doPost()` showing direct rendering of unfiltered database records in `details.html`.

```
try {
    if (authenticated(username, password)) {
        // Get search results and merge with template
        Map<String, Object> model = new HashMap<>();
        model.put(key: "records", searchResults(surname));
        Template template = fm.getTemplate("details.html");
        template.process(model, response.getWriter());
    }
}
```

Flaw 5 - Missing Session Management / Weak Access Control

Nature of the Flaw

The application does not create or validate HTTP sessions. After login, the server never records the authenticated user state. Protected pages can be accessed by directly sending POST requests to `/details`.

How it was Discovered

`doPost()` calls `authenticated(username, password)` and immediately renders `details.html` without `request.getSession()` or stored session attributes.

Why it is a Security Issue

Without session management:

- Any attacker can replay a previously captured POST request.
- Requests without credentials can access patient data if they include a surname.
- There is no persistent authentication between requests.

This breaks the entire security model and makes unauthorised access trivial.

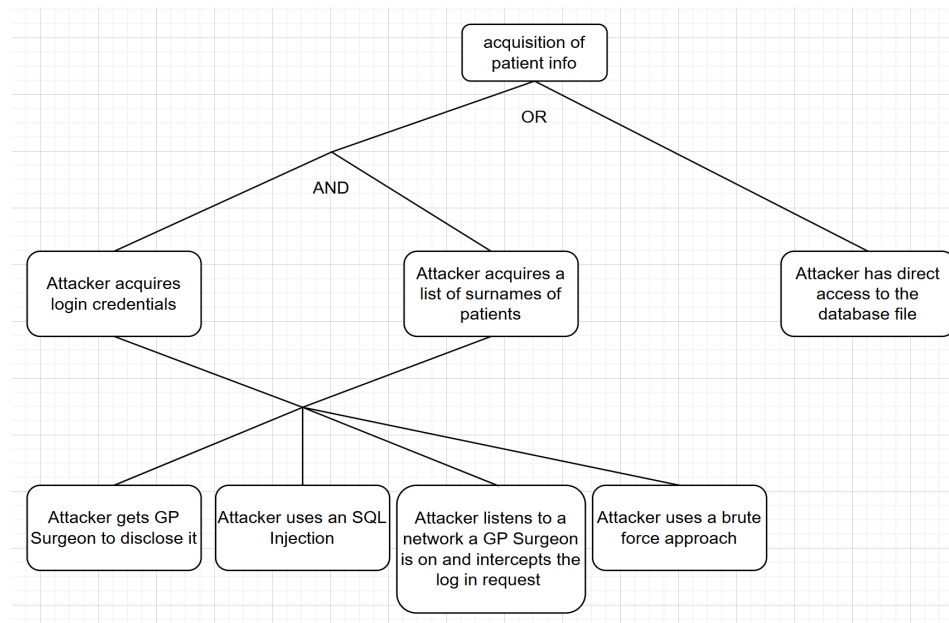
Evidence: Figure 5

```
try {
    if (authenticated(username, password)) {
        // Get search results and merge with template
        Map<String, Object> model = new HashMap<>();
        model.put(key: "records", searchResults(surname));
        Template template = fm.getTemplate(name: "details.html");
        template.process(model, response.getWriter());
    }
    else {
        Template template = fm.getTemplate(name: "invalid.html");
        template.process(dataModel: null, response.getWriter());
    }
}
```

Task 2: Visualisation

Attack Tree:

The following attack tree illustrates how an attacker can exploit the vulnerabilities identified in Task 1 to obtain the system's primary asset: patient medical information. The root node represents the attacker's end goal, while intermediate nodes model the logical steps (AND/OR relationships) required to reach that objective.



Note:

- When one or more edges split into more edges, it means that all the boxes above the split are connected to all the boxes after it.

Explanation of the Attack Tree

The attack tree models two high-level attack strategies for compromising the confidentiality of patient records:

1. Direct compromise through database access (OR branch)

An attacker may obtain patient information by gaining direct access to the SQLite database file.

This branch is enabled by **Flaw 1 (plaintext passwords)** and **Flaw 2 (SQL injection)**, since plaintext credentials or injected queries can reveal the file path or stored data. If they deployment environment exposes the database file (e.g., poor filesystem permissions), the attacker can bypass the web application entirely.

2. Web-based compromise requiring two conditions (AND branch)

To extract patient information via the web interface, the attacker must obtain both:

1. Valid GP login credentials, and
2. A list of patient surnames

These two requirements combine under an AND relationship because both must be satisfied for the attacker to query the patient database through the application.

2.1 Obtaining GP login credentials

This node is supported by multiple attack paths enabled by identified flaws:

- Social engineering a GP: A GP might be tricked into revealing their credentials (leveraging Flaw 1, as passwords are plaintext and easy to communicate).
- SQL Injection: Exploits **Flaw 2**, allowing extraction of stored credentials or bypassing authentication altogether.
- Intercepting a login request: Feasible due to **Flaw 5 (no session management)** - the system transmits login attempts without secure session enforcement or protections against replay.
- Brute forcing credentials: Possible because the application performs no lockout or rate-limiting.

These four sub-nodes are in an OR relationship, as any one of them allows the attacker to obtain a usable login.

2.2 Acquiring Patient Surnames

Surnames are used as search parameters in the application. Due to **Flaw 3 (broken access control)** and **Flaw 2**, any surname can retrieve all matching records. An attacker can gather surnames from:

- Intercepting a GP's request,
- Brute-force enumeration of common surnames,
- SQL injection that reveals all surnames.

Once both credentials and surnames are acquired, the attacker can repeatedly query the system and reconstruct full patient records.

Assumptions

- The attacker has network access to the web interface (e.g., local attacker, shared network, or remote access if deployed externally).
- No HTTPS, encryption, or request integrity protections are in place (supported by the missing session management in Flaw 5).
- GPs rely on simple passwords (supported by findings in Task 1).
- The system stores all patient information in a single SQLite file located on the server.

How the Flaws Support the Attack Tree

Each flaw directly maps to a node in the attack tree:

- **Flaw 1 (Plaintext password storage)**: Enables credential theft if the database is accessed/
- **Flaw 2 (SQL Injection)**: Enables authentication bypass, data extraction, and retrieval of entire patient tables.
- **Flaw 3 (Broken access control)**: Allows any authenticated attacker to access all patients once credentials are acquired.
- **Flaw 4 (Unsanitised output)**: Allows injection of malicious payloads to compromise GP accounts, facilitating credential theft.
- **Flaw 5 (No session management)**: Enables replay of login requests and unauthorised access to protected endpoints.

Together, these flaws create several realistic paths for an attacker to reach the root node: **acquisition of patient information**.

Task 3: Implementation of Security Fixes

Fix Identification

Flaw 1 - Plaintext Password Storage

The attack tree shows multiple attack paths that depend on the attacker acquiring valid GP credentials—through SQL injection, social engineering, intercepted login requests, or brute-force attempts. Because the database stored passwords in plaintext, any partial database compromise would immediately leak valid login credentials.

To remove this high-impact attack path, all stored passwords were replaced with **bcrypt salted hashes**, and password verification was moved to **BCrypt.checkpw()**. This ensures credentials cannot be recovered even if the database is accessed, eliminating the “attacker acquires GP credentials” branch of the attack tree.

Flaw 2 - SQL Injection Vulnerabilities

SQL injection appeared as a distinct branch of the attack tree, enabling attackers to bypass authentication and extract the entire patient table. Since the application constructed SQL queries using string concatenation, attackers could manipulate the query structure to return arbitrary data.

Flaw 3 - Broken Access Control (GPs Can View Any Patient)

Another attack tree branch assumed that once an attacker acquired a single GP login (legitimately or through compromise) they could access all patient records by supplying arbitrary surnames. This was possible because the system lacked GP-specific access restrictions. To close this branch, patient searches were modified to require the authenticated GP’s ID, enforcing `WHERE surname = ? AND gp_id = ?`. This ensures that even a compromised GP account cannot be used to access other doctors’ patient data, removing the “attacker escalates privileged access after login” path.

Flaw 4 - Unsanitised Data Rendering (Stored XSS Risk)

The attack tree included a propagation branch where an attacker who compromises a GP account could inject malicious scripts into patient fields, enabling further takeover when other GPs view those records. This was possible because the application rendered database fields without sanitisation. To break this escalation path, Freemarker HTML escaping (`?html`) was added to `details.html` so all patient information is safely encoded before rendering. This prevents stored XSS payloads from executing in the browser, cutting off the attack path where injected scripts enable wider compromise.

Flaw 5 - Missing Session Management / Weak Access Control

A major branch of the attack tree involved bypassing authentication by replaying POST requests or accessing protected pages without a valid login, enabled by the absence of server-side session tracking. Without sessions, the server trusted any request containing a surname. To eliminate this branch, proper **server-side session management** was introduced: a session is created upon successful authentication, the GP’s ID is stored in the

session, and every protected request now validates the session before processing. This breaks the “attacker replays request → server returns patient information” branch entirely.

Together, these fixes were chosen specifically because they dismantle all viable attacker routes identified in the attack tree, ensuring that no straightforward path remains for an attacker to obtain patient information.

Fixes Implemented

This section summarises the changes made to the application to correct each of the five vulnerabilities identified in Task 1. For each flaw, the modifications to the codebase and database are briefly described, together with an explanation of how the fix removes the underlying security weakness. All changes were tested to ensure the application remained functional and no longer exposed the original vulnerabilities.

Flaw 1 - Plaintext Password Storage

What was changed

- All plaintext passwords in db.sqlite3 were replaced with bcrypt salted hashes (cost factor 10).
- The authentication logic in AppServlet.java was redesigned so that the SQL query retrieves only the stored hash for a given username.
- Password verification is now performed using BCrypt.checkpw() instead of SQL-level password comparison.

How this fixes the flaw

This prevents attackers from recovering valid GP credentials even if the database is accessed through SQL injection, local file compromise, or misconfiguration. Because hashes are one-way and salted, a database leak no longer results in credential theft, removing the attack path where a stolen password leads directly to patient data access.

Flaw 2 - SQL Injection Vulnerabilities

What was changed

- All SQL queries in AppServlet.java were updated to use PreparedStatement with parameter binding.
- The authentication query now uses WHERE username = ?, and the password is checked in Java rather than SQL.
- The patient search query was rewritten as SELECT * FROM patient WHERE surname = ? AND gp_id = ?.

How this fixes the flaw

Prepared statements prevent attacker-controlled input from altering SQL syntax, eliminating the possibility of injecting conditions such as ' OR '1'='1'. This fix blocks authentication bypass, mass data extraction, and tampering with WHERE clauses, thereby removing all SQL-injection branches from the attack tree.

Flaw 3 - Broken Access Control (GPs Can View Any Patient)

What was changed

- The authentication function was modified to return the authenticated GP's doctor_id rather than a boolean.
- doPost() now stores this ID in the session and passes it to the search function. searchResults() was rewritten to accept both the surname and the GP's ID.
- The SQL query was updated to enforce per-GP filtering using WHERE surname = ? AND gp_id = ?.

How this fixes the flaw

The system now enforces that each GP only sees records associated with their own ID. Even if an attacker logs in using stolen credentials, they cannot access other doctors' patient records. This fully eliminates the privilege-escalation issue where any authenticated GP could enumerate all patients.

Flaw 4 - Unsanitised Data Rendering (Stored XSS Risk)

What was changed

- The details.html template was updated to apply Freemarker's HTML escaping operator (?html) to every field displayed from the database.
- All dynamic patient fields (name, date of birth, diagnosis, GP ID, notes) are now encoded before being inserted into the HTML output.

How this fixes the flaw

Escaping prevents embedded scripts or HTML from executing in the browser. Even if an attacker manages to insert a malicious payload into the database, it will be displayed safely as text rather than executing. This stops stored XSS propagation and ensures that viewing a patient record cannot compromise a GP session.

Flaw 5 - Missing Session Management / Weak Access Control

What was changed

- A new HTTP session is created upon successful login using request.getSession(true).
- The authenticated GP's ID is stored in the session and used for all subsequent operations.
- Protected endpoints now validate the session; requests without a valid session are redirected to the login page.
- Failed logins invalidate any existing session to prevent session fixation.
- A session timeout was configured (e.g., 15 minutes) to limit exposure from unattended sessions.

How this fixes the flaw

The server now maintains a reliable authentication state, ensuring that protected pages cannot be accessed through replayed POST requests or direct URL submission. Patient data is only returned when a valid authenticated session is active, eliminating the branch where attackers bypass authentication entirely.

Overall Impact

These fixes collectively remove all practical paths for attackers to reach the root goal of the attack tree: gaining access to patient information. Authentication, session management, SQL handling, data sanitisation, and privilege controls were all strengthened, and the application now enforces industry-standard security practices throughout its execution flow.